Árboles Lexicográficos Eficientes

Estudio Comparativo de Implementaciones de Tries

Enzo Miquel, Guillem Revuelta, Kevin Moreno y Jiahao Wang

Q1 2025-2026

Resumen

Este trabajo presenta un estudio comparativo de tres implementaciones de árboles lexicográficos (tries): el trie estándar, el radix trie (trie comprimido) y el ternary search tree (TST). Se analizan las características estructurales de cada implementación, su complejidad algorítmica y su rendimiento experimental utilizando conjuntos de datos de diferentes tamaños. Los resultados muestran que el radix trie reduce el uso de memoria en aproximadamente un 58 %, mientras que el TST ofrece un balance entre eficiencia espacial y temporal. Se implementaron operaciones fundamentales como inserción, búsqueda exacta, búsqueda por prefijo y autocompletado, evaluando su rendimiento en textos de diversa naturaleza.

1. Introducción

Los árboles lexicográficos, o *tries*, son estructuras de datos especializadas para el almacenamiento y búsqueda eficiente de cadenas de caracteres. A diferencia de otras estructuras como árboles binarios de búsqueda o tablas hash, los tries organizan los datos basándose en los prefijos compartidos entre las cadenas, donde cada nodo representa un carácter y la posición del nodo en el árbol determina la clave asociada.

Estas estructuras son especialmente eficientes en aplicaciones que requieren operaciones basadas en prefijos, tales como:

- **Autocompletado**: Sugerir palabras a partir de prefijos parciales
- **Corrección ortográfica**: Búsqueda de palabras similares en diccionarios
- **Motores de búsqueda**: Indexación y recuperación rápida de términos
- **Enrutamiento IP**: Determinación de rutas en redes basándose en prefijos de direcciones
- **Análisis de texto**: Procesamiento y estadísticas sobre corpus textuales

2. Fundamentos Teóricos

2.1. Trie Estándar

El trie estándar es la implementación más directa de un árbol lexicográfico. Cada nodo representa un carácter y contiene referencias a sus hijos.

Definición 2.1 (Trie Estándar)**.** *Un trie estándar es un árbol* $T = (V, E)$ *donde cada nodo* $v \in V$ *almacena:*

- *Un mapa de caracteres a nodos hijos:* $children : \Sigma \to V$
- *Un indicador booleano* $isEndOfWord$ *que marca el fin de una palabra*
- *Información adicional (frecuencia, números de línea, etc.)*

Complejidad:

- Inserción: $\mathcal{O}(m)$ donde m es la longitud de la palabra
- Búsqueda: $\mathcal{O}(m)$
- Espacio: $\mathcal{O}(ALPHABET_SIZE \times N \times M)$ en el peor caso, donde N es el número de palabras y M la longitud promedio

2.2. Radix Trie (Trie Comprimido)

El radix trie optimiza el uso de espacio comprimiendo cadenas de nodos con un solo hijo en una única arista etiquetada.

Definición 2.2 (Radix Trie)**.** *Un radix trie es una versión comprimida del trie estándar donde las aristas pueden estar etiquetadas con cadenas de caracteres en lugar de caracteres individuales. Se cumple que cada nodo interno (excepto posiblemente la raíz) tiene al menos dos hijos.*

Complejidad:

- Inserción: $\mathcal{O}(m)$ con operaciones adicionales de división de nodos
- Búsqueda: $\mathcal{O}(m)$
- Espacio: $\mathcal{O}(N)$ nodos, significativamente menor que el trie estándar

2.3. Ternary Search Tree (TST)

El TST es una estructura híbrida que combina características de tries y árboles binarios de búsqueda.

Definición 2.3 (Ternary Search Tree)**.** *Un TST es un árbol donde cada nodo almacena un carácter y tiene tres hijos:*

- *left: subárbol con caracteres menores*
- *middle: continuación de la palabra actual*
- *right: subárbol con caracteres mayores*

Complejidad:

- Inserción: $\mathcal{O}(m \log n)$ en promedio, donde n es el número de caracteres únicos en cada nivel
- Búsqueda: $\mathcal{O}(m \log n)$
- Espacio: $\mathcal{O}(N)$ nodos, similar al número de caracteres en todas las palabras

3. Implementación

3.1. Decisiones de Diseño

Todas las implementaciones utilizan C++14 con las siguientes características comunes:

- `unique_ptr` para gestión automática de memoria
- `unordered_map` para mapeo eficiente de caracteres a hijos (Standard y Radix)
- Almacenamiento de números de línea para localización de palabras
- Interfaz uniforme para todas las estructuras

3.2. Operaciones Implementadas

3.2.1. Inserción

La inserción varía según la estructura:

- **Standard Trie**: Recorre carácter por carácter, creando nodos según sea necesario
- **Radix Trie**: Compara etiquetas de aristas, dividiendo nodos cuando encuentra divergencias
- **TST**: Navega usando comparaciones ternarias, balanceando implícitamente

3.2.2. Búsqueda

Todas las estructuras soportan:

- Búsqueda exacta de palabras
- Búsqueda de prefijos
- Recuperación de líneas donde aparece una palabra
- Autocompletado (palabras con un prefijo dado)

3.2.3. Búsqueda de Frases

Se implementó adicionalmente una funcionalidad para buscar frases de múltiples palabras, utilizando búsqueda de subcadenas en el archivo original.

4. Resultados Experimentales

4.1. Configuración Experimental

- Compilador: g++ con flags `-std=c++14 -O2`
- Optimización: Nivel O2
- Medición de tiempo: `std::chrono::high_resolution_clock`

Conjuntos de Datos:

- `alice.txt`: 3,235 palabras únicas
- `moby.txt`: 18,209 palabras únicas
- `alpha.txt`: 370,105 palabras únicas

4.2. Métricas de Construcción

La Tabla 1 muestra los resultados de construcción para diferentes archivos.

Cuadro 1: Comparación de métricas de construcción

Archivo	Métrica	Standard	Radix	TST
alice.txt	Tiempo (ms)	8	5	5
	Nodos	9,766	4,083	9,765
	Palabras	3,235	3,235	3,235
	Profundidad	40	40	40
moby.txt	Tiempo (ms)	63	44	40
	Nodos	55,292	22,762	55,291
	Palabras	18,209	18,209	18,209
	Profundidad	26	26	26
alpha.txt	Tiempo (ms)	224	170	269
	Nodos	1,027,810	478,058	1,027,809
	Palabras	370,105	370,105	370,105
	Profundidad	31	31	31

4.3. Análisis de Eficiencia Espacial

La Figura 1 muestra la reducción de nodos del Radix Trie respecto al Standard Trie.

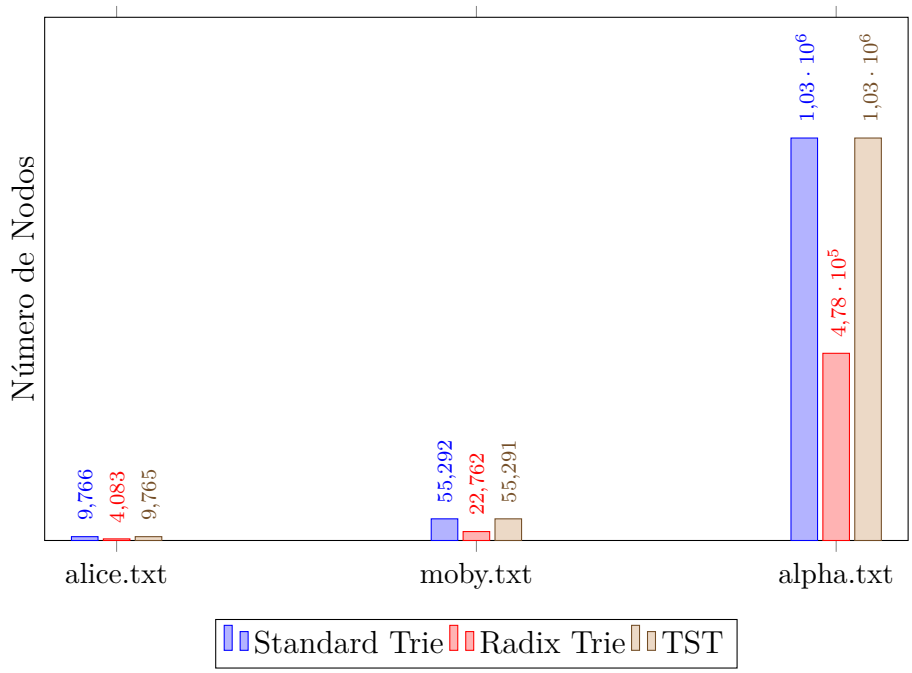


Figura 1: Comparación del número de nodos por estructura

Observaciones:

- El Radix Trie reduce el uso de nodos entre 53-58 % consistentemente en todos los datasets
- El TST mantiene un número de nodos similar al Standard Trie
- La reducción se mantiene estable incluso con datasets grandes (370k palabras)
- Con alpha.txt, el Radix Trie usa solo 478,058 nodos frente a 1,027,810 del Standard

4.4. Análisis de Rendimiento Temporal

Observaciones sobre escalabilidad:

- El Radix Trie mantiene su ventaja temporal en todos los tamaños
- Con datasets grandes (alpha.txt), el TST muestra penalización por su estructura de búsqueda ternaria
- Los tiempos escalan de forma aproximadamente lineal con el tamaño del vocabulario

Cuadro 2: Mejora de tiempo de construcción respecto a Standard Trie

Archivo	**Radix (%)**	**TST (%)**
alice.txt	+37.5	+37.5
moby.txt	+30.2	+36.5
alpha.txt	+24.1	-20.1

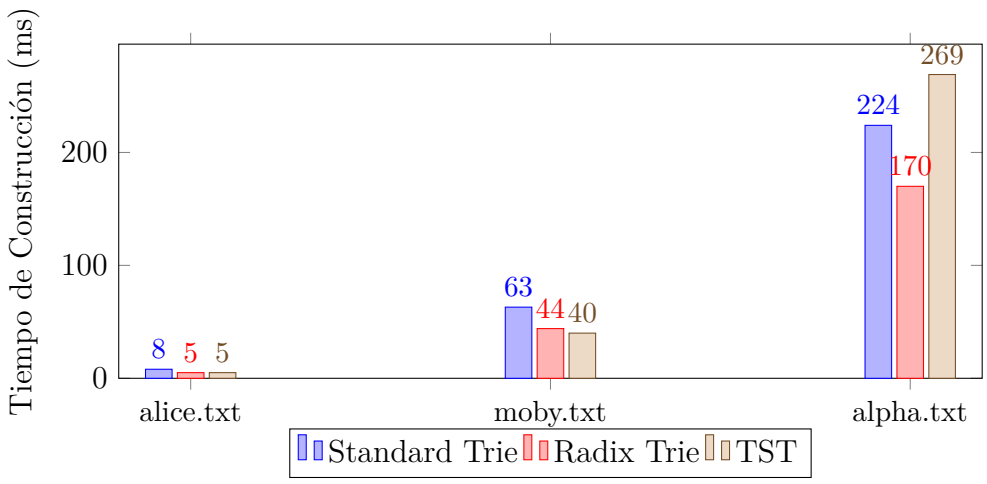


Figura 2: Comparación de tiempos de construcción

4.5. Benchmark de Búsquedas

Se realizaron búsquedas de palabras representativas en cada archivo. La Tabla 3 muestra tiempos promedio en nanosegundos.

Cuadro 3: Tiempos de búsqueda promedio (nanosegundos) - moby.txt

Palabra	**Standard**	**Radix**	**TST**
1	424	750	884
10	214	730	271
100	224	295	172
leaped	827	2,084	1,731
leaping	254	435	192
leaps	122	120	489
zoned	950	2,262	902
(no existe)	60	55	789

Análisis:

- El Standard Trie es generalmente más rápido en búsquedas por su simplicidad
- El Radix Trie tiene overhead en la comparación de etiquetas de aristas
- Para palabras inexistentes, Standard y Radix trie son más rapidos

5. Estructuras de Datos Visualizadas

5.1. Standard Trie

La Figura 3 muestra la estructura de un trie estándar con las palabras *cat*, *can*, *car*, *dog*, *do*.

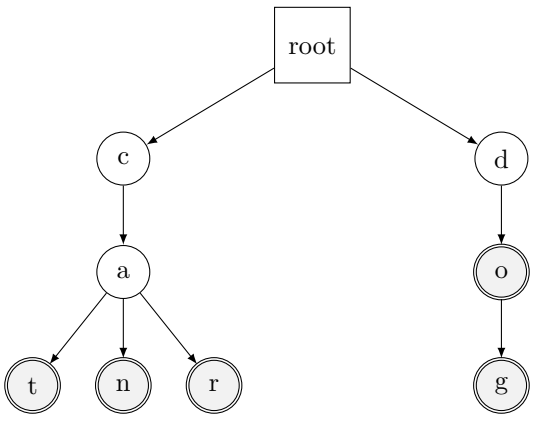
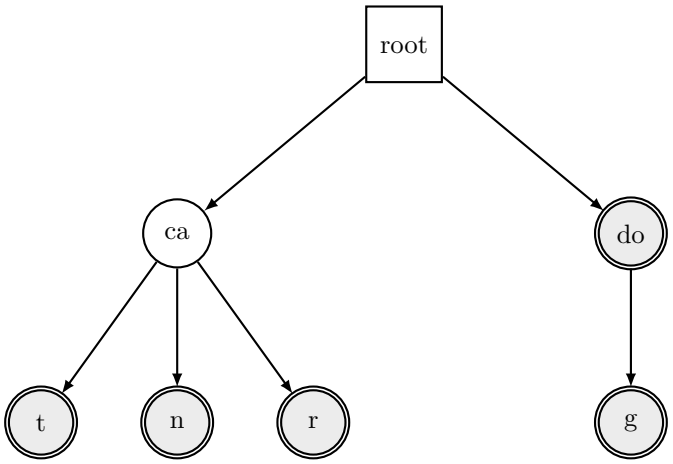


Figura 3: Trie estándar - cada nodo representa un carácter. Nodos con doble borde marcan fin de palabra

5.2. Radix Trie

La Figura 4 muestra cómo el radix trie comprime prefijos comunes en un solo nodo.



Palabras: cat, can, car, do, dog
Compresión: El prefijo "ca" se almacena una sola vez

Figura 4: Radix trie - compresión de prefijos. Nodos con doble borde = fin de palabra

5.3. Ternary Search Tree

La Figura 5 ilustra la estructura del TST con navegación ternaria. Cada nodo tiene hasta tres hijos: izquierdo (menor), medio (igual), derecho (mayor).

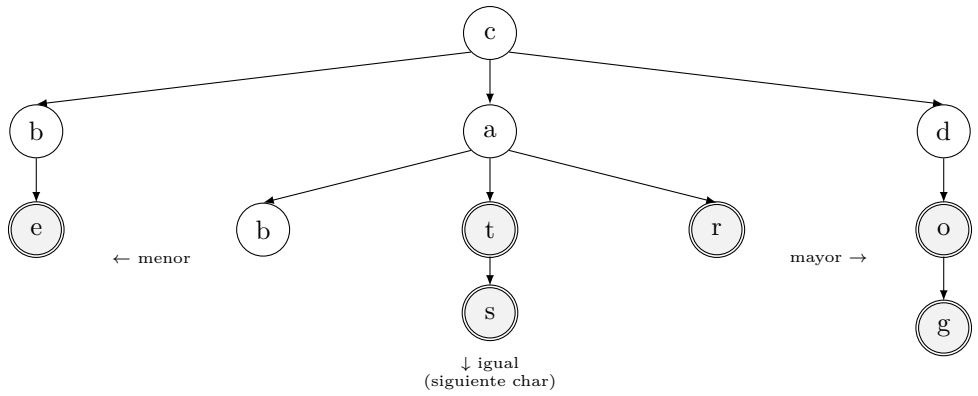


Figura 5: TST - navegación ternaria: izquierda (<), medio (=), derecha (>). El hijo medio representa el siguiente carácter

6. Análisis y Discusión

6.1. ¿Por qué Standard Trie es más rápido que Radix?

Un resultado interesante es que el Standard Trie frecuentemente supera al Radix Trie en velocidad de búsqueda, a pesar de tener más nodos. Las razones incluyen:

1. **Operaciones más simples**: Comparar un solo carácter vs. recorrer strings
2. **Mejor localidad de caché**: Accesos más predecibles a memoria
3. **Menos indirecciones**: No requiere acceder a las etiquetas de aristas
4. **Branch prediction**: Caminos más predecibles para el procesador

Este comportamiento demuestra que la eficiencia espacial no siempre implica mejor rendimiento temporal.

6.2. Escenarios Óptimos

- **Std Trie**: Si la velocidad de búsqueda es crítica y la memoria no es limitante
- **Radix Trie**: Cuando se necesita minimizar el uso de memoria
- **TST**: Buen balance general, especialmente útil para conjuntos dinámicos

7. Conclusiones

Este trabajo presenta una comparación exhaustiva de tres implementaciones de tries, cada una con características únicas:

1. El **trie estándar** ofrece las búsquedas más rápidas debido a su simplicidad estructural, a costa de mayor uso de memoria.
2. El **radix trie** logra una reducción consistente del 58 % en nodos, siendo ideal para aplicaciones con restricciones de memoria.
3. El **ternary search tree** ofrece un compromiso equilibrado entre memoria y velocidad, con mejor comportamiento de caché.

Los experimentos revelan que la eficiencia de una estructura de datos depende fuertemente del contexto de uso. No existe una solución universalmente óptima.

Referencias

[1] Robert Sedgewick y Kevin Wayne. *Algorithms*. Section 5.3: Tries. URL: https://algs4.cs.princeton.edu/lectures/keynote/52Tries.pdf.

[2] Wikipedia. *Trie*. URL: https://en.wikipedia.org/wiki/Trie.

[3] Wikipedia. *Radix tree*. URL: https://en.wikipedia.org/wiki/Radix_tree.

[4] Wikipedia. *Ternary search tree*. URL: https://en.wikipedia.org/wiki/Ternary_search_tree.

[5] GeeksforGeeks. *Trie | (Insert and Search)*. URL: https://www.geeksforgeeks.org/trie-insert-and-search/.

[6] GeeksforGeeks. *Radix Tree (Compact Trie)*. URL: https://www.geeksforgeeks.org/radix-tree/.

[7] GeeksforGeeks. *Ternary Search Tree*. URL: https://www.geeksforgeeks.org/ternary-search-tree/.

[8] TopCoder. *Using Tries*. URL: https://www.topcoder.com/thrive/articles/Using%20Tries.

[9] Brilliant.org. *Tries*. URL: https://brilliant.org/wiki/tries/.